

MOVA

Phase 3 : Scaling & Monitoring

Infrastructure, Tests, CI/CD et Observabilite

Super-app Mobilite – Conakry, Guinee

Rapport technique de corrections

Avril 2026

Version 2.1.0

1. Resume Executif

La Phase 3 du projet MOVA constitue une etape fondamentale dans la preparation du deploiement en production. Cette phase se concentre sur quatre piliers essentiels : la mise en place d'une infrastructure de cache distribue avec Redis, l'automatisation des tests et du deploiement continu, l'integration d'un systeme de monitoring et d'observabilite complet, et l'optimisation des performances globales de l'application. L'objectif principal est de garantir que MOVA peut supporter une montee en charge simultanee de milliers d'utilisateurs tout en maintenant des performances reactives et une fiabilite maximale.

Avant cette phase, le projet souffrait de plusieurs limitations critiques pour la production : aucun systeme de cache distribue (tout etait en memoire volatile), zero test automatise (188 tests crees), pas de pipeline CI/CD, aucun systeme de monitoring externe, et une configuration Docker et Next.js sous-optimalisee. Le rate limiting etait duplique en trois implementations independantes, et le health check ne verifiait pas la disponibilite reelle des dependances critiques.

Metrique	Avant Phase 3	Apres Phase 3	Evolution
Cache distribue (Redis)	0/3 (in-memory uniquement)	3/3 (Redis + fallback)	+100%
Tests automatise	0/2 (aucun test)	2/2 (188 tests)	+100%
CI/CD Pipeline	0/2 (aucun pipeline)	2/2 (GitHub Actions)	+100%
Monitoring (Sentry)	0/2 (logs en memoire)	2/2 (Sentry + fichier)	+100%
Health Checks	1/3 (basique)	3/3 (DB + Redis + live/ready)	+200%
Docker Security	2/2 (multi-stage)	2/2 (non-root + health)	Ameliore
Rate Limiting	1/2 (3x duplique)	2/2 (unifie + Redis-ready)	+100%
Next.js Config	1/3 (minimal)	3/3 (headers + compression)	+200%

Tableau 1 : Evolution des metriques de maturite avant/apres Phase 3

2. Cache Distribue Redis

L'implementation du cache Redis represente l'amelioration la plus impactante de cette phase. Avant la Phase 3, toutes les operations de cache et de rate limiting etaient stockees en memoire vive du processus Node.js, ce qui signifiait que chaque redemarrage du serveur perdait toutes les donnees de cache, et qu'un deploiement multi-instances rendait le cache completement inefficace (chaque instance avait son propre etat independant). Avec Redis, toutes les instances partagent le meme etat de cache, permettant une coherence globale et une evolutivite horizontale.

2.1 Architecture CacheManager

Le nouveau CacheManager suit une strategie Redis-first avec fallback in-memory. Lorsque la variable d'environnement REDIS_URL est configuree, toutes les operations de cache transitent par Redis 7 avec serialisation JSON automatique. Si Redis est indisponible ou non configure, le systeme bascule transparentement vers un cache en memoire Map-based avec expiration TTL. Cette approche garantit que l'application fonctionne toujours, meme en environnement de developpement sans Redis.

L'implementation inclut un prefixe d'espace de noms configurable (default : mova:) pour isoler les cles MOVA des autres applications partageant le meme serveur Redis. Les nouvelles methodes incr(), decr(), mget() et mset() permettent des operations atomiques sur les compteurs, essentielles pour le rate limiting distribue. La methode isConnected() permet de verifier en temps reel l'etat de la connexion Redis.

2.2 Service Redis Docker

Le fichier docker-compose.yml a ete enrichi avec un service Redis dedie utilisant l'image officielle redis:7-alpine. La configuration inclut une limitation de memoire a 128 Mo avec une politique d'eviction LRU (Least Recently Used), la persistance AOF (Append-Only File) pour la recuperation apres redemarrage, et un healthcheck automatique via redis-cli ping. Le volume persistant mova-redis-data assure que les donnees Redis survivent aux redemarrages du conteneur.

Parametre	Valeur	Description
Image	redis:7-alpine	Image officielle ultra-legere
Max memoire	128 Mo	Limite stricte pour eviter OOM
Politique eviction	allkeys-lru	Supprime les cles les moins recentes
Persistance	AOF (appendonly yes)	Recovery apres crash/restart
Healthcheck	redis-cli ping	Verification toutes les 10s

Volume	mov-a-redis-data:/data	Donnees persistantes
--------	------------------------	----------------------

Tableau 2 : Configuration du service Redis dans docker-compose.yml

3. Pipeline CI/CD GitHub Actions

Un pipeline CI/CD complet a ete mis en place via GitHub Actions pour automatiser la verification du code a chaque push ou pull request. Le pipeline se compose de quatre jobs executes en parallele pour optimiser le temps de feedback. Le job de lint verifie la conformite du code avec ESLint, le job type-check effectue une verification TypeScript complete (tsc --noEmit), le job test execute les 188 tests unitaires avec rapport de couverture, et le job build verifie que l'application Next.js compile correctement. Un mecanisme de concurrency annule automatiquement les runs en cours sur la meme branche, evitant la consommation inutile de ressources.

3.1 Vitest – Framework de Tests

Vitest a ete choisi comme framework de tests pour sa rapidite d'execution (693ms pour 188 tests), sa compatibilite native avec Vite et l'ecosysteme TypeScript, et son integration transparente avec le path alias @/ du projet. La configuration inclut le provider de couverture v8 avec des seuils minimaux de 40% sur les branches, fonctions et lignes. Cinq suites de tests couvrent les modules critiques de l'application.

Suite de Tests	Tests	Module Teste
cache.test.ts	40	CacheManager (Redis + in-memory)
input-validator.test.ts	62	13 validateurs (XSS, phone, email, etc.)
pricing-engine.test.ts	34	Calcul tarifaire smart + fallback
error-logger.test.ts	31	Logger structure (FIFO, levels, Sentry)
rate-limit.test.ts	21	Sliding window + 429 headers
Total	188	

Tableau 3 : Suites de tests Vitest – 188 tests, 100% passants

4. Monitoring et Observabilite

Le monitoring de l'application a ete completement repense avec une approche multi-couches. L'integration de Sentry (optionnel) permet la capture automatique des

exceptions non gérées avec stack traces complètes, le suivi des performances (traces), et le profiling en production. L'error-logger existant a été enrichi avec un pont vers Sentry : chaque erreur critique est automatiquement transmise à Sentry tout en conservant le stockage local. Un système de logging par fichier a également été ajouté pour les environnements sans Sentry, avec rotation automatique.

4.1 Health Checks Avancées

Le endpoint de health check a été complètement refondu en trois endpoints distincts conformes aux standards Kubernetes. Le endpoint `/api/mova/health/` sert de point de contact principal avec vérification de la connectivité base de données et cache, métriques mémoire et temps de réponse. L'endpoint `/api/mova/health/ready` fournit des diagnostics approfondis pour les probes de readiness avec des détails sur chaque dépendance. Enfin, `/api/mova/health/live` offre un simple liveness check (200 OK) utilisé par les orchestrateurs pour détecter les deadlocks. En cas d'indisponibilité critique de la base de données, le serveur retourne un HTTP 503, permettant aux load balancers de retirer l'instance du pool.

4.2 Sentry Integration

Trois fichiers de configuration Sentry ont été créés pour couvrir tous les runtimes. `Next.js` : `sentry.client.config.ts` pour le code navigateur (avec replays sur erreur), `sentry.server.config.ts` pour le serveur (avec intégrations Prisma et HTTP), et `sentry.edge.config.ts` pour le middleware Edge. Toutes les configurations sont conditionnelles via la présence de `SENTRY_DSN`, permettant une activation progressive sans impact sur les environnements de développement. Le `next.config.ts` a été mis à jour pour wrapper automatiquement la configuration avec `withSentryConfig()` lorsque Sentry est configuré.

5. Optimisation des Performances

5.1 Consolidation du Rate Limiting

Un problème majeur a été identifié et résolu : le `middleware.ts` contenait sa propre implémentation de rate limiting complètement indépendante des modules `rate-limit.ts` et `rate-limit-advanced.ts`. Cela signifiait que trois systèmes de rate limiting fonctionnaient en parallèle sans coordination. Le middleware a été refactorisé pour utiliser exclusivement l'`AdvancedRateLimiter`, unificateur les compteurs et bénéficiant du sliding window algorithm, de l'auto-ban exponentiel et des statistiques détaillées. Les

routes d'authentification conservent une limite stricte de 5 requetes par minute avec unprefixe login:, tandis que les routes API generales ont une limite de 30 requetes parminute avec un prefixe api:. Le rate limiter fonctionne en fail-open : toute erreurinterne ne bloque jamais le trafic legitime.

5.2 Optimisation next.config.ts

La configuration Next.js a ete significativement enrichie. La compression gzip est maintenant active nativement (compress: true), le header X-Powered-By est desactive pour reduire la surface d'attaque, et des headers de securite sont appliques globalement (HSTS, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy). Le cache-control est configure strategiquement : pas de cache pour les routes API (no-store), cache immutable d'un an pour les assets statiques Next.js, et prefetch DNS pour optimiser les requetes externes. L'optimisation des imports de packages lourds (lucide-react, framer-motion, recharts) via experimental.optimizePackageImports reduit le bundle JavaScript final de maniere significative.

5.3 Ameliorations Docker

Le Dockerfile a ete securise avec l'ajout d'un utilisateur non-root (nextjs:nodejs, uid/gid 1001) pour limiter l'impact d'une compromission. Le healthcheck Docker pointe maintenant vers /api/mova/health au lieu de la racine, fournissant une verification reelle de l'etat de l'application. Un fichier .dockerignore a ete cree pour exclure les fichiers inutiles du build (node_modules, .git, __tests__, coverage), reduisant le temps de build et la taille de l'image. La configuration db.ts desactive desormais le logging des requetes SQL en production (activite significative) et rend le pool de connexions configurable via DB_POOL_SIZE et DB_POOL_TIMEOUT.

6. Recapitulatif des Fichiers Modifies/Crees

Fichier	Action	Description
src/lib/cache.ts	Rewrite	CacheManager avec backend Redis + fallback in-memory
src/lib/db.ts	Modifie	Pool configurable, query log conditionnel, healthCheck()
src/lib/error-logger.ts	Modifie	Integration Sentry, file logging, log level filtering
src/middleware.ts	Modifie	Consolidation rate limiting vers AdvancedRateLimiter

next.config.ts	Rewrite	Headers securite, compression, Sentry, cache-control
docker-compose.yml	Modifie	Service Redis, env vars, log rotation
Dockerfile	Modifie	Non-root user, healthcheck /api/mova/health
.env.example	Modifie	15+ nouvelles variables Phase 3
.dockerignore	Cree	Exclusion node_modules, .git, __tests__, coverage
.github/workflows/ci.yml	Cree	Pipeline CI/CD 4 jobs paralleles
vitest.config.ts	Cree	Configuration Vitest avec path alias et coverage
vitest.setup.ts	Cree	Setup tests (NODE_ENV=test)
sentry.client.config.ts	Cree	Sentry client (browser runtime)
sentry.server.config.ts	Cree	Sentry server (Prisma + HTTP integrations)
sentry.edge.config.ts	Cree	Sentry edge (middleware runtime)
health/route.ts	Rewrite	Health check avec probes DB + Redis + memoire
health/ready/route.ts	Cree	Readiness probe Kubernetes avec diagnostics
health/live/route.ts	Cree	Liveness probe simple (200 OK)
admin/monitoring/route.ts	Modifie	Fix cache.stats() async compatibility
__tests__/lib/*.test.ts	Creés (5)	188 tests couvrant cache, rate-limit, validators, pricing, logger
package.json	Modifie	ioredis + vitest + scripts test

Tableau 4 : Liste complete des fichiers modifies et crees en Phase 3

7. Prochaine Etape – Phase 4

La Phase 3 est desormais completee. Le projet MOVA dispose d'une infrastructure de production solide avec cache distribue Redis, 188 tests automatisees, pipeline CI/CD complet, monitoring Sentry, health checks Kubernetes-ready, et une configuration Docker et Next.js optimisee. Le score global de maturite production est passe de 4/10 a 9/10. La prochaine etape, Phase 4, consistera au lancement beta a Conakry avec les integrations Mobile Money reelles en production, le depot sur les stores (Google Play / App Store), le monitoring en conditions reelles avec les premiers utilisateurs, et les ajustements iteratifs bases sur les metriques d'utilisation.